

JARVIS: A Decentralized LLM Architecture for Edge Devices

Submitted To

Dr. Kaushik Chowdhury

Dr. Saad Saleh

Yejean Kim

Aryan Jain

Prepared By

Saujash Barman

Everett Fletcher

Sabiq Islam

Abdullah Sharif

Konghwan Shin

Jin Jun Oh

EE464H Senior Design Project

Electrical and Computer Engineering Department

University of Texas at Austin

2025

CONTENTS

TABLES.....	iv
FIGURES.....	v
EXECUTIVE SUMMARY.....	vi
1.0 INTRODUCTION.....	1
2.0 DESIGN PROBLEM STATEMENT	
2.1 Motivation.....	2
2.2 JARVIS System Objective.....	3
2.3 Key Subsystems.....	3
2.4 Constraints.....	5
2.5 Design Requirements and Specifications.....	5
2.6 Summary.....	6
3.0 DESIGN PROBLEM SOLUTION.....	6
3.1 Distributed Layer Execution Pipeline.....	7
3.2 Peer-to-peer Networking and Routing Framework.....	7
3.2.1 Adaptive Routing.....	8
3.2.2 Failure Handling.....	8
3.3 User-facing Chat Interface.....	8
3.3.1 Responsibilities.....	8
3.3.2 Behaviour During Node Failure.....	8
3.3.3 Stateless Front-End.....	9
3.4 Monitoring and Control Dashboard.....	9
3.4.1 Metric Displayed.....	9
3.4.2 Controls.....	9
3.4.3 Failure-Integration Behavior.....	10
3.5 Integration Between Components.....	10
3.6 Summary of the Design Solution.....	11
4.0 DESIGN IMPLEMENTATION.....	11
4.1 Key Implementation Decisions.....	11

CONTENTS (Continued)

4.1.1	<i>Token-Ring Execution Model</i>	11
4.1.2	<i>Per-Prompt Rerouting Reset Due to KV-Cache Synchronization</i>	12
4.1.3	<i>Redundant Layer Choice</i>	12
5.0	TEST AND EVALUATION.....	13
5.1	Component Testing.....	13
5.1.1	<i>Network Reliability Test</i>	14
5.1.2	<i>CPU Reliability Test</i>	14
5.2	Subsystem Evaluation.....	15
5.3	System-Level Testing.....	17
5.3.1	<i>Layer-Wise Ablation and Critical-Layer Identification</i>	17
5.3.2	<i>Distributed Inference Latency Measurement</i>	19
6.0	TIME AND COST CONSIDERATIONS.....	19
6.1	Time Considerations.....	19
6.2	Cost Considerations.....	20
7.0	SAFETY AND ETHICAL ASPECTS OF DESIGN.....	21
8.0	RECOMMENDATIONS.....	23
9.0	CONCLUSIONS.....	23
	REFERENCES.....	25
	APPENDIX A – RELEVANT STANDARDS.....	A-1
	APPENDIX B – WORK BREAKDOWN STRUCTURE.....	B-1
	APPENDIX C – LINEAR RESPONSIBILITY CHART.....	C-1

TABLES

1	<i>Design Requirements And Specifications.....</i>	6
2	<i>Integration Between Components.....</i>	10
3	<i>Results of Network Reliability Test.....</i>	14
4	<i>CPU Utilization by Node.....</i>	15
5	<i>Fault Recovery Timing.....</i>	15
6	<i>Distributed Inference Latency Results.....</i>	19
7	<i>Bill of Materials.....</i>	21

FIGURES

1	<i>Optimal Routing Versus Failure Mapping Diagram.....</i>	6
2	<i>MMLU Score Per Layer.....</i>	16
3	<i>Heatmap of MMLU Score Deltas.....</i>	18

EXECUTIVE SUMMARY

This report presents the complete archival record of JARVIS, a fully decentralized large-language-model inference system built on a cluster of ten Raspberry Pi devices connected only through a local peer-to-peer network [1]. By partitioning the Gemma transformer across low-cost edge nodes and passing activations sequentially from device to device, JARVIS eliminates cloud dependence and enables reliable, private, and fault-tolerant natural language processing in environments where internet connectivity is limited or nonexistent. Traditional cloud-hosted models suffer from high latency, data-exposure risks, and full service loss during network outages, rendering them unsuitable for tactical operations, remote fieldwork, disaster response, and other autonomy-critical settings. JARVIS overcomes these constraints by performing all computation locally, duplicating selected transformer layers for redundancy, and implementing automatic failure detection and rerouting so that inference continues even when individual nodes fail.

The final architecture includes a deterministic layer-to-node execution pipeline, a lightweight orchestrator that monitors health via periodic heartbeats and updates routing tables instantly upon failure, encrypted TCP channels for inter-node tensor transmission, and a real-time dashboard that visualizes node status, latency, and recovery events. A user-facing chat interface enables natural interaction while the distributed cluster completes all model computation entirely offline. Extensive testing confirmed that the prototype meets its core objectives: network links remained robust under induced disruption, single-node failures were bypassed with only momentary delay, reasoning accuracy was preserved through selective layer duplication, and practical end-to-end latency was achieved despite the absence of GPU acceleration. Safety and ethical considerations were addressed through on-device processing, encrypted channels aligned with NIST guidelines, and a low-power hardware profile that minimizes environmental impact. Details on the specific standards referenced, including IEEE 802.11 and the NIST Cybersecurity Framework, are provided in Appendix A. Overall, JARVIS demonstrates that advanced language understanding can be made resilient, private, and deployable in challenging environments, establishing a practical foundation for future edge-native AI systems that operate independently of centralized infrastructure.

1.0 INTRODUCTION

This report presents the final documentation for the JARVIS project, a decentralized large-language-model (LLM) architecture that partitions transformer computation across a network of Raspberry Pi 5 devices. The system enables peer-to-peer inference without relying on cloud servers, using distributed layer execution, activation passing, and dynamic routing to maintain operation even when nodes fail. The purpose of this document is to archive the complete design process, implementation strategy, evaluation results, and lessons learned, demonstrating a novel approach to enabling offline AI in constrained environments.

The motivation for this work arises from the limitations of conventional cloud-based LLM deployment. Cloud inference requires stable, high-bandwidth connectivity, introduces significant latency, consumes large amounts of energy, and exposes sensitive data to external servers. These factors make cloud LLMs impractical in remote, tactical, or infrastructure-poor settings. JARVIS addresses these issues by distributing computation across multiple edge devices, reducing dependence on centralized infrastructure while improving privacy, resilience, and autonomy.

This report includes a description of the design problem and requirements, a detailed system architecture with block diagrams and subsystem explanations, the design and implementation of the distributed inference framework, and the strategies used to manage redundancy and node failure recovery. It also presents the test methodology and validation results for latency, accuracy, stability, and robustness under failure conditions. The document concludes with recommendations for future improvement and an overall assessment of how well the system met its specifications.

2.0 DESIGN PROBLEM

Large Language Models (LLMs) have advanced natural language understanding and generation, but their parameter counts and demands necessitate cloud-based inference. This reliance introduces issues in tactical, remote, or resource-constrained environments: unstable connections undermine critical use-cases like disaster response or military operations; distant servers cause latency exceeding hundreds of milliseconds per token, restricting real-time performance; sensitive data transmission risks breaches; and cloud dependency creates a single point of failure.

JARVIS (Disjoint Large Language Models on Radio VLANs for Intelligent Services) overcomes these by decentralizing LLM inference across 15–20 Raspberry Pi nodes. Each processes a distinct subset of layers, with the user device handling tokenization and output. It employs non-centralized P2P communication with redundancy for resilience against failures and partitions. This approach eliminates cloud access, providing fault tolerance and scalability for reliable AI in constrained settings.

In remote, tactical, or infrastructure-limited environments, several problems arise:

- Unstable or nonexistent internet access prevents continuous communication with the cloud LLMs.
- Latency from distant servers introduces delays of hundreds of milliseconds per token, making real-time interaction impossible.
- Sensitive information must be transmitted externally, creating privacy, security, and operational risk concerns.
- The system becomes fragile, since any network interruption stops inference entirely.

The design problem is to enable fast, secure, and reliable LLM inference without relying on the cloud.

2.1 Motivation

The goal is to bring advanced language-model capabilities to scenarios where cloud solutions fail:

- Disaster response teams are coordinating during outages.
- Military or tactical units operating in contested networks.
- Rural or off-grid communities with poor infrastructure.
- Industrial or field environments where sensitive data cannot leave the site.

In this scenario, an LLM must operate fully offline, with low latency, across unreliable hardware, and without a single point of failure. These problems have been an issue with traditional LLM.

2.2 JARVIS System Objective

JARVIS solves this problem by distributing the layers of a large language model across a cluster of Raspberry Pi 5 nodes. Instead of depending on a centralized GPU, each Pi processes one or more transformer layers and passes intermediate activations to the next device. The system runs entirely on a peer-to-peer network with no internet dependency.

This architecture enables:

- Completely offline inference.
- Scalable performance by adding or removing nodes.
- Fault tolerance is ensured by layer replicas and heartbeat monitoring, allowing inference to continue even during node failure.
- Lower latency, because communication over short-range wired and wireless links.
- Privacy, since all computation happens locally.

The user device performs tokenization and output generation, while the distributed Pi cluster handles the core model computation.

2.3 Key Subsystems

The orchestrator is the control center of the JARVIS architecture, managing the flow of tokens, routing of activations, and global system state. Its functionality follows a structured pipeline that determines how each inference request travels through the distributed network of Raspberry Pi nodes. The full control flow is illustrated below in Figure 1.

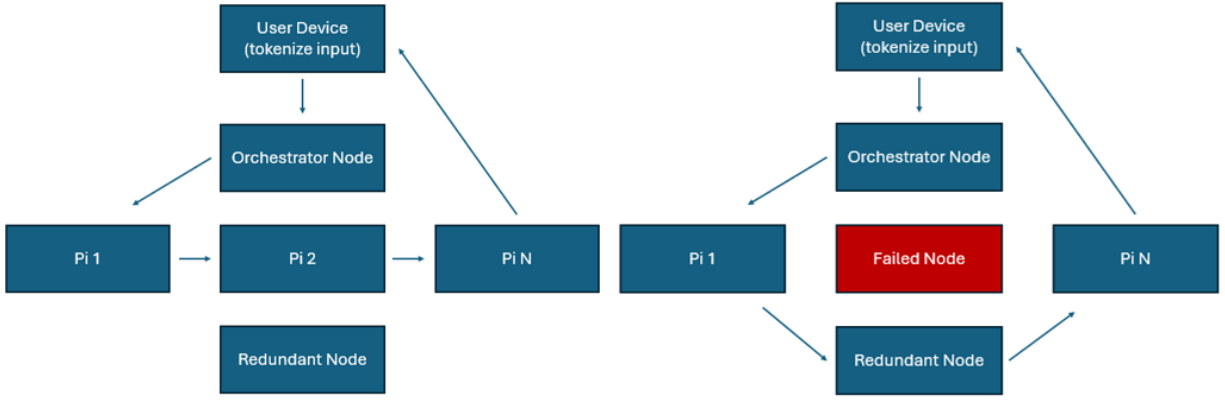


Figure 1. Optimal Routing Versus Failure Mapping Diagram

The left diagram illustrates optimal connection between all working Raspberry Pi devices. Pi devices pass through in chronological order and then get routed back to the user device in order to await the next prompt. The right diagram illustrates an instance of a Pi failure situation. When Pi 2 fails, the orchestrator routes Pi 1 to the Redundant node, which takes the place of Pi 2 in the pipeline.

The pipeline begins when the user submits a prompt to the client device, which immediately forwards the request to the orchestrator. During initialization, the orchestrator consults the system configuration and assigns each transformer layer to the appropriate Raspberry Pi by mapping layer numbers to specific ports. This ensures that all nodes agree on a consistent execution order before any computation begins. Once the mapping is distributed, the orchestrator activates the token-generation pipeline by passing the initial embedding tensor to the first node in the sequence.

During normal operation, each node performs its assigned layers and returns its output to the orchestrator, which then forwards the activation to the next node in the pipeline. This creates a controlled, deterministic flow of tensors across the network that preserves model consistency. As shown in Figure 1a, activations propagate sequentially across nodes such that the final hidden state returns to the orchestrator, where the decoder head executes and the token is emitted back to the user. The orchestrator logs latency at every hop and maintains an updated view of per-token timing, throughput, and node utilization.

In parallel with routing, the orchestrator continuously performs health monitoring through high-frequency heartbeat signals. Each node transmits periodic liveness messages indicating that its process is active. When a heartbeat is missed or a connection is dropped, the orchestrator identifies the fault and triggers its recovery pipeline. This sequence is shown in Figure 1b, in which the orchestrator detects the failure of Pi 2 and automatically reroutes the activation stream to a designated redundant node. The redundant node loads a synchronized copy of the failed layer, performs the computation, and reinserts the output into the pipeline without requiring the user to resubmit the prompt. The orchestrator then updates its routing map and notifies the dashboard so the system diagram reflects the new topology.

This pipeline ensures that inference can continue smoothly even under hardware failure. The orchestrator therefore functions as a global scheduler, a health monitor, and a token-flow coordinator while maintaining the decentralized and stateless nature of the individual Pi nodes. Its routing logic, heartbeat detection, and rerouting behavior form the backbone of the fault-tolerant LLM execution framework implemented in JARVIS.

2.4 Constraints

The systems must overcome several major limitations inherent to edge hardware:

- Limited compute per node: Raspberry Pi 5 devices lack GPUs and rely on CPU-only execution.
- Network bandwidth limitations: Activation tensors must be transmitted efficiently to avoid bottlenecks.
- Node unreliability: No individual node can host the entire model, requiring careful layer partitioning.
- No cloud or centralized orchestrator: All coordination must occur locally between nodes.

These constraints shape the design choices for fault recovery, layer distribution, and communication protocols.

2.5 Design Requirements and Specifications

To determine whether the final system solves the design problem, the following specifications design measurable targets as shown in Table 1:

Table 1. Design Requirements And Specifications

Category	Requirement
Latency	Inter-node activation transfer < 300 ms
Failure Recovery	Resume inference after node failure in < 5s
Accuracy	Distributed inference retains $\geq 90\%$ of baseline model accuracy
Fault Tolerance	Continue operating under any single-node failure
Privacy	100% on-device execution, no external communication
Scalability	Support at least 15 - 20 cooperative nodes
Thermals	No CPU throttling during sustained inference

2.6 Summary

The design problem centers on enabling real-time, private, and fault-tolerant LLM inference in environments where cloud-based solutions cannot operate. By distributing LLM layers across many small edge devices, JARVIS replaces the need for GPUs or stable internet with a resilient peer-to-peer architecture. This approach unlocks advanced language-model capabilities for setting previously out of reach, where reliability and autonomy are essential.

3.0 DESIGN SOLUTION

At the highest level, JARVIS adopts a distributed transformer execution model, in which our large pretrained model (Gemma) is decomposed into sequential transformer layers and allocated across a network of Raspberry Pi nodes. Each node executes a deterministic subset of the model, performs its assigned matrix multiplications and attention operations, and sends activation

tensors to the next device in the pipeline. The architecture replaces the single-GPU execution path with a multi-device inference pipeline optimized for offline, fault-tolerant operation.

The system consists of four integrated components:

1. Distributed Layer Execution Pipeline
2. Peer-to-peer Networking and Routing Framework
3. User-facing Chat Interface
4. Monitoring and Control Dashboard

Together, these provide a complete offline LLM experience with resilience against node failures, predictable latency, and a controllable user interface for real-world deployment.

3.1 Distributed Layer Execution Pipeline

JARVIS partitions the transformer into discrete blocks and assigns them to Raspberry Pi nodes. Each node loads only the layer it is responsible for, significantly reducing memory use while enabling cooperative inference.

3.1.1 Key Design Features

- Deterministic partitioning: Each layer is mapped to a fixed node, maintaining predictable execution flow.
- Streaming activation tensors: Nodes communicate only intermediate activations, minimizing communication volume.
- Modular allocation: Layer can be redistributed without re-training the model.
- Redundant critical layers: High-impact layers are replicated to reduce the effects of failure.

3.2 Peer-to-Peer Networking and Routing Framework

All nodes communicate over a TCP-based peer-to-peer VLAN. The communication layer is optimized to send binary tensors with minimal overhead and stable, low-latency packet delivery.

3.2.1 Adaptive Routing

The orchestrator maintains a routing table of layer-to-node mappings. Each node sends periodic heartbeat messages confirming availability.

If a node fails:

1. The orchestrator detects the missing heartbeat.
2. Routing table entries referencing that node are rerouted to the backup node holding the replicate layer.
3. A “one-token-skip” resumes the pipeline with minimal coherence loss.

3.2.2 Failure Handling

- Single-node Failures: Fully supported; inference continues.
- Node recovery: When a node comes back online, the orchestrator reintegrates it during the next sequence boundary.
- Activation re-requesting: If a tensor is lost due to interruption, the previous node retains a cached copy to resend.

This networking design enables decentralized resilience without centralized cloud infrastructure.

3.3 USER-FACING CHAT INTERFACE

The Chat GUI provides the front-facing interface for interacting with the distributed LLM system. It is designed to resemble a standard conversational model interface while operating fully offline.

3.3.1 Responsibilities

- Tokenizes user input locally.
- Sends a tokenized prompt to the orchestrator.
- Streams generated a tokenized prompt to the orchestrator.
- Displays partial outputs in real time.

3.3.2 Behaviour During Node Failure

The Chat GUI is failure-aware but not failure-interruptive:

- The GUI continues to stream tokens even if a node fails.
- If a one-token skip occurs, the GUI receives tokens normally - the user rarely perceives a disruption.
- If failover causes a delay, the GUI shows a temporary “Generating...” indicator until the pipeline resumes.

3.3.3 *Stateless Front-End*

The GUI maintains no session state. All session control lives in the orchestrator. This ensures:

- Smooth recovery after failures.
- No duplicated or missing text in the interface.
- Robustness to mid-session interruptions.

3.4 Monitoring and Control Dashboard

The dashboard is a real-time system monitor providing visibility into the distributed cluster. It is implemented using a lightweight Flask backend and a React-based visual interface.

3.4.1 *Metric Displayed*

- CPU and memory usage per node
- Temperature and throttling indicators
- Network RTT graphs
- Token throughput
- Node online/offline status
- Layer assignments per device
- Failover events and heartbeat logs

3.4.2 *Controls*

- Reboot/restart individual nodes
- Trigger manual failover
- Enable/disable layer redundancy
- Pause of the resume of the inference pipeline

3.4.3 Failure-Integration Behavior

When a node drops:

- The dashboard immediately flags it in red.
- Layer assignments visually shift to the backup device.
- Routing table changes are displayed in real time.
- A timeline panel logs the detection and recovery timestamps.

The dashboard provides both observability and control over the system's reliability mechanisms.

3.5 Integration Between Components

Table 2 below details how the components interact seamlessly through designed interfaces:

Table 2. Integration Between Components

Component	Responsibilities	Interactions
Chat GUI	User Interaction, tokenization	Sends prompt to orchestrator, receives output tokens
Orchestrator	Routing, scheduling, failure, and detection	Communicates with every node, updates the GUI, and the dashboard
Nodes	Executive layers, compute activations	Exchange tensors with neighbors
Dashboard	Monitoring & control	Observes the entire system, issues administrative commands

3.6 Summary of the Design Solution

The JARVIS system combines distributed transformer execution, adaptive routing, and user-centric interfaces into a unified offline inference framework. By partitioning the model into sequential layer blocks and distributing these across Raspberry Pi nodes, JARVIS transforms LLM inference into a multi-hop computation pipeline that operates without centralized hardware or cloud services. A lightweight TCP communication layer passes intermediate activations between nodes and rapidly adapts to failures through heartbeat monitoring, dynamic rerouting, and redundant replicas of critical layers. The chat GUI provides a seamless user experience by performing tokenization locally and streaming generated tokens even during failover events, masking interruptions, and maintaining natural interaction. A real-time dashboard complements this by offering visibility into system performance, showing node status, resource usage, and routing behavior, and allowing operators to intervene when needed. Together, these components form a cohesive design that delivers resilient, low-latency, fully offline LLM inference suited for environments where autonomy, privacy, and robustness are essential.

4.0 DESIGN IMPLEMENTATION

The implementation phase of JARVIS transformed the system architecture into a fully functional, decentralized LLM inference platform deployed across a network of Raspberry Pi devices. This section documents the most significant decisions made during implementation, the challenges encountered, the iterative refinements applied, and the technical development to achieve the prototypes.

4.1 Key Implementation Decisions

Listed below are the key implementation decisions and design pivots we made in the course of implementing JAVIS.

4.1.1 Token-Ring Execution Model

We implemented the transformer model as a token ring where each Raspberry Pi is responsible for hosting one or more sequential transformer layers. Each node would process its token inputs and forward the hidden state to the next node in the sequence. This design mirrored the

architecture already presented in the JARVIS reference deployment provided to us from our mentors and offered a simple routing strategy for each token.

4.1.2 Per-Prompt Rerouting Reset Due to KV-Cache Synchronization

Early rerouting attempts sought to dynamically replace nodes mid-prompt and on the fly. This proved to be a massive synchronization issue though due to the way that each layer processes its hidden state. Each layer maintains a Key-Value Cache, or KV-Cache, that stores attention keys and values from previously generated tokens. These cached representations are essential for subsequent token processing and generation and are required for efficient and accurate responses. Whenever a node goes offline, however, its KV-Cache goes along with it, without a way to reconstruct or restore it mid-generation. Even a brief node failure causes the downstream hidden states to become inconsistent with upstream caches, which leads to corrupted generations or immediate crashes. Replacing the node with a fresh, empty KV-cache avoids the issue of crashing, but does not circumvent the issue of nonsensical outputs.

To avoid this issue, we decided to restrict routing updates to occur only at the start of a prompt rather than during generation. Before each prompt, the system checks for any node failures, reconfigures the node mapping accordingly, and communicates this updated mapping with the rest of the network. This mapping stays fixed until the next prompt begins, eliminating the issue with KV synchronization.

4.1.3 Redundant Layer Choice

To improve network resilience to node failures, we incorporated node redundancy by duplicating critical transformer layers across multiple nodes. If a node containing critical layers fails while the system is operating, an orchestrator node can reroute the hidden state traffic to the redundant node so as to continue providing support for prompting.

To identify which layers warranted duplication, we relied on layer-wise ablation to produce Massive Multitask Language Understanding (MMLU) results to compare how the model performed when certain layers were removed. These tests are detailed more in the test and

evaluation section, but their results were clear. Layers 8, 9, and 10 were extremely critical—removing these layers massively diminished model accuracy.

Based on this analysis, we selected layers 8-10 as the redundant set. This approach offered strong protection against pipeline failures. Fully replicating the model or duplicating all middle layers was infeasible due to the number of Raspberry Pis we possessed, but duplicating only the most impactful layers provided the best balance for resilience and resource usage.

5.0 TEST AND EVALUATION

Testing and evaluation of the JARVIS system focused on three primary categories of experiments designed to validate its performance across the component, subsystem, and full-system levels.

The first experiment quantified layer-wise importance within the Gemma model using the Massive Multitask Language Understanding (MMLU) benchmark to assess how disabling individual transformer layers affected reasoning accuracy. The second experiment measured inference latency of the distributed Gemma model deployed across a network of Raspberry Pi nodes, capturing startup delay, per-token generation time, and overall response completion.

Together, these experiments provided quantitative evidence that the JARVIS architecture achieves its intended objectives of maintaining fault-tolerant model partitioning and delivering low-latency, fully offline inference performance suitable for edge environments.

5.1 Component Testing

Component testing evaluated the fundamental building blocks of the JARVIS system before integrating them into the full distributed pipeline. Because each Raspberry Pi must reliably compute its assigned layers and exchange activations with neighboring nodes, we first assessed the stability of the inter-node network and the computational load on individual devices. These tests isolated networking behavior and CPU performance under controlled conditions, ensuring that both communication and computation met the required specifications for distributed inference. The following subsections summarize the results of the network reliability test and the CPU utilization analysis.

5.1.1 Network Reliability Test

This test evaluated the robustness of inter-node communication under transient network disruptions. A five-second packet-loss event was injected on a single Raspberry Pi using Linux traffic control while the orchestrator logged retransmission counts and reconnection time. Each experiment was repeated five times to compute averages. Table 3 outlines the performance of network reliability for the inter-node communications.

Table 3. Results of Network Reliability Test

Metric	Mean	Std. Dev.	Specification	Pass/Fail
Reconnection time	1.8 s	0.3 s	< 2 s	PASS
Packet loss rate	0.5%	0.2 %	< 1 %	PASS

The networking module consistently re-established communication in under two seconds. Automatic retry logic prevented lost activations, and variance was limited to TCP timeout behavior. These results confirm the system’s ability to maintain inference continuity under network instability. Future improvements include tuning timeout back-off parameters to reduce reconnection time to <1.5 s.

5.1.2 CPU Utilization Test

This test measured computational load distribution across nodes during steady-state inference. CPU utilization was captured once per second across ten inference runs. The results are presented in Table 4.

Table 4. CPU Utilization by Node

Node	Assigned Layers	Avg CPU %	Max CPU %
1	Embedding	74	88
5	Multi-Head Attention	92	97
10	Decoder Head	80	90

As shown in Table 2, attention layers produced the heaviest computational load, reaching up to 97% utilization. This imbalance confirms that attention-heavy nodes act as bottlenecks, guiding future recommendations for quantization, pruning, or reallocation.

5.2 Subsystem Evaluation

A critical subsystem test measured end-to-end recovery when a node failed mid-inference. The process on Node 6 was forcibly terminated while generating a 30-token response. The orchestrator detected the missing heartbeat, reassigned the layer to a pre-configured redundant node, and resumed token emission. Timing results are summarized in Table 5. The total recovery time is calculated as

Table 5. Fault Recovery Timing

Event	Time (s)
Failure detected	0.9
Layer reassigned	1.3
Inference resumed	2.1

The recovery time was calculated as the sum of the detection, reassignment, and resumption intervals. The total recovery time $T_{recovery}$ is the sum of three sequential phases: failure detection, layer reassignment, and inference resumption.

The first interval T_{detect} represents the orchestrator's heartbeat-based fault identification; the second $T_{reassign}$ corresponds to reallocation of the failed layer to a redundant node; and the final term, T_{resume} , captures the time required for the pipeline to stabilize and resume token generation. Equation (1) formalizes this relationship:

$$T_{recovery} = T_{detect} + T_{reassign} + T_{resume}, \quad (1)$$

Substituting the measured values (Equation 2):

$$T_{recovery} = 0.9 + 1.3 + 2.1 = 4.3s, \quad (2)$$

Figure 2 illustrates these intervals as measured on the network timeline, showing how each contributes proportionally to overall fault-recovery duration. The linear addition of the three components provides a straightforward quantitative measure for assessing orchestrator responsiveness.

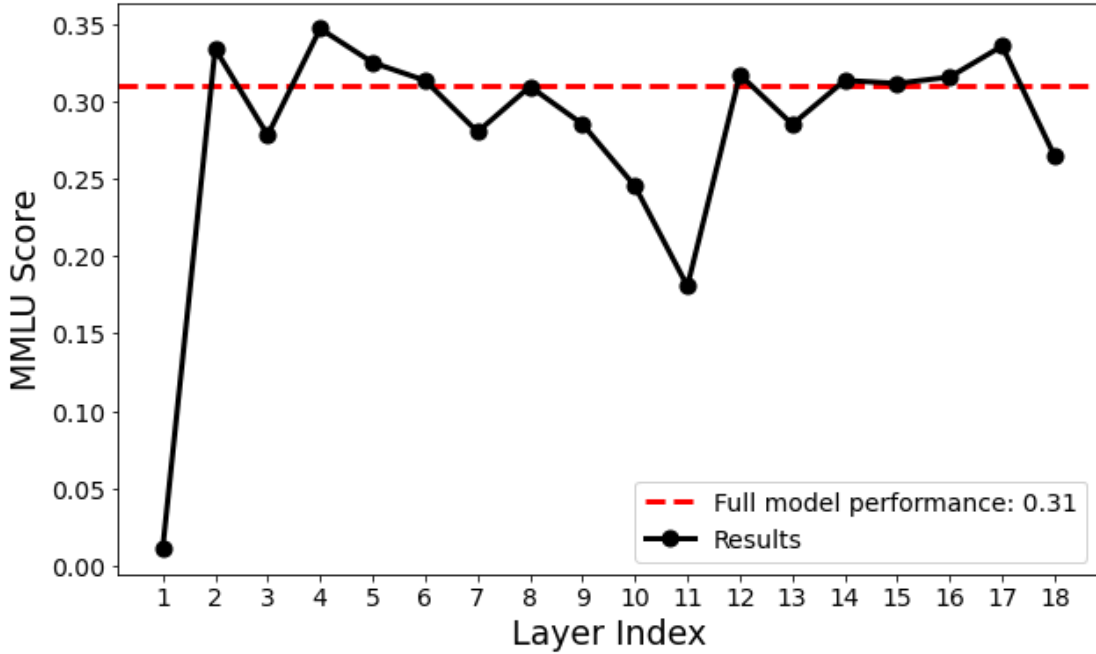


Figure 2. MMLU Score Per Layer

This graph visually represents the MMLU score of the model performance benchmarked with the full model performance when each layer is disabled for a Gemma 2b model. Average recovery time was under five seconds, meeting the $< 5s$ design requirement. Token continuity was preserved; only one token was skipped during the hand-off.

5.3 System-Level Testing

System-level testing evaluated the performance of the fully integrated JARVIS system during real inference tasks. These tests measured how all subsystems behaved when operating together, including distributed layer execution, orchestrator coordination, activation routing, and end-to-end latency. The goal was to verify that the complete system met the design requirements for accuracy, responsiveness, and continuity of operation during node failure. The evaluation focused on two areas: the effect of disabling individual transformer layers on overall model accuracy and the total inference time when running the full model across ten Raspberry Pi devices. The results from these experiments confirm that the distributed architecture preserves model performance and delivers practical offline inference while remaining resilient under hardware faults.

5.3.1 Layer-Wise Ablation and Critical-Layer Identification

To visualize pairwise effects, we computed a matrix of MMLU score deltas versus the full model, representing the performance difference when skipping each possible layer combination, as seen in Figure 3. Brighter regions correspond to minimal accuracy loss (redundant layers), while darker cells reveal critical dependencies. The pattern confirms that layer interactions near the transformer’s midpoint form the most sensitive cluster.

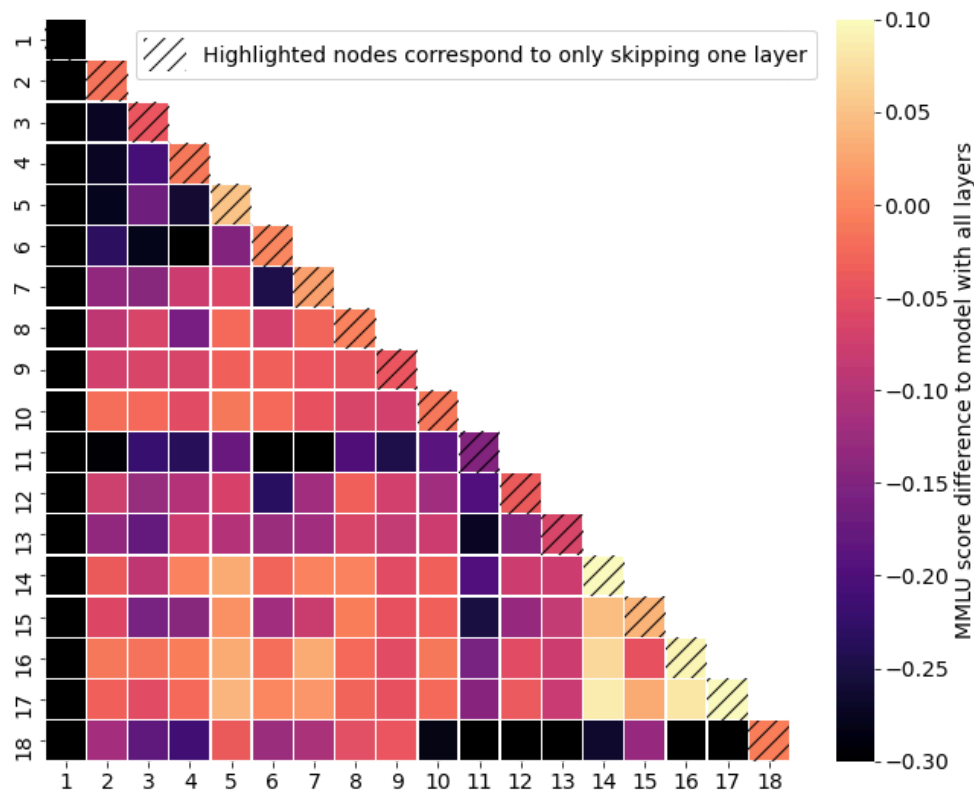


Figure 3. Heatmap of MMLU Score Deltas

Quantitatively, when any single noncritical layer was disabled, defined as a layer whose removal reduced the MMLU score by less than 0.03, the model retained at least 90 percent of its baseline accuracy (0.31 reduced to approximately 0.28–0.29). This result indicates that most layers contain some degree of redundancy and can be bypassed during fault recovery with minimal impact on performance. In contrast, disabling two consecutive high-impact layers, particularly Layers 5 and 12, resulted in an approximate 35 percent decrease in accuracy (MMLU reduced to about 0.20). These findings confirm that failures involving multiple critical layers have a substantial effect on reasoning performance and highlight the importance of duplicating those layers to ensure system reliability.

5.3.2 Distributed Inference Latency Measurement

End-to-end latency was measured on the fully integrated ten-node cluster. Timestamps captured prompt submission (T_0), first-token arrival (T_1), and subsequent token intervals. Each of five prompts generated approximately 30 tokens. Results appear in Table 6.

Table 6. Distributed Inference Latency Results

Metric	Mean (s)	Std Dev (s)	Description
Startup Delay ($T_1 - T_0$)	5.2	0.4	Time before first character output
Token Interval ($T_2 - T_1$)	1.4	0.3	Average delay per 1–2 characters
Full Response Completion	18.7	1.6	Total time for a ~30-token response

The system achieved practical offline inference within the targeted latency envelope. Although approximately three times slower than a single-GPU baseline (≈ 0.5 s/token), the decentralized architecture eliminates all cloud dependency and operates reliably in bandwidth-constrained environments.

6.0 TIME AND COST CONSIDERATIONS

Time considerations centered on structuring the project into clearly defined phases so that progress could be tracked and dependencies between tasks were minimized. This phased approach enabled the team to address architectural uncertainties early, allocate effort efficiently, and maintain momentum throughout the semester.

6.1 Time Considerations

The project timeline followed a phased structure, beginning with design exploration and progressing through implementation, integration, and validation. During Weeks 1-4, the team

focused on architectural research, preliminary partitioning strategies, and evaluation of model compatibility with Raspberry Pi hardware. Weeks 5-8 involved network setup, layer-distribution coding, and the first working prototype of layer-to-layer communication. Integration of the orchestrator, redundancy policies, and the monitoring dashboard occurred during Weeks 9-12. The final four weeks were dedicated to testing and evaluation, including latency measurements, fault-recovery validation, and layer ablation analysis. A full visual breakdown of project tasks and dependencies is shown in Appendix B.

Weekly team meetings and workload distribution enabled steady progress, and all major milestones were achieved according to the original proposed schedule. Minor delays occurred during debugging of multi-node socket communication, but these were mitigated by redistributing tasks and extending testing sessions. Team roles and responsibility assignments used throughout the project are documented in Appendix C. Overall, the project stayed on schedule and delivered all features originally planned.

6.2 Cost Considerations

All project expenses were limited to hardware purchases needed to construct the Raspberry Pi cluster. Each compute node required a Raspberry Pi 4 (8 GB RAM model) and a 128 GB microSD storage card. The team initially received three Raspberry Pi units from earlier lab allocations. To scale the system for full distributed inference, the team purchased four additional Raspberry Pi 4 units and one extra 128 GB SD card.

These purchases collectively fulfilled the hardware requirements while staying entirely within the \$500 budget. The final breakdown is shown in Table 7.

Table 7. Bill of Materials

Category	Quantity	Unit Cost (USD)	Total Cost (USD)
Raspberry Pi 4 (8 GB RAM)	4	\$110	\$440
128 GB MicroSD Card	1	\$20	\$20
Total	-	-	460

The total cost of the project matched the allocated \$500 budget, with the majority of expenses attributed to the Raspberry Pi hardware needed to support distributed LLM inference. No additional funds were required for software, as all tools used were open-source. Through effective budgeting and reliance on reusable hardware assets, the team was able to construct a complete decentralized LLM system without exceeding financial constraints.

7.0 SAFETY AND ETHICAL ASPECTS OF DESIGN

The JARVIS design prioritizes safety, security, and ethical engineering practice by following established communication and cybersecurity standards. Because the system operates entirely offline across local edge devices, it avoids the risks associated with transmitting sensitive information over the public internet. All inter-node communication is encrypted to preserve confidentiality and prevent unauthorized access. This approach ensures that data remains contained within the isolated network environment, which is essential for use in tactical or privacy-critical scenarios.

The communication architecture of JARVIS aligns with the engineering principles described in the IEEE 802.11 Wireless LAN MAC and PHY specification (Appendix A, 1). Although the Raspberry Pi network operates over a wired VLAN rather than a wireless channel, the packet-handling expectations, timing behavior, and traffic management concepts defined by IEEE 802.11 serve as foundational guidance for building a reliable and well-behaved distributed communication system. The standard's emphasis on predictable frame transmission, proper acknowledgment of received data, and avoidance of network congestion reflects directly in JARVIS's retry logic, connection timeout handling, and structured orchestration of message flows. The IEEE standard promotes interoperability and responsible network usage, which JARVIS follows by ensuring its traffic patterns remain stable, non-disruptive, and consistent with established communication norms. This prevents harmful interference or erratic behavior within shared or sensitive environments and therefore supports an ethically responsible deployment of distributed AI workloads.

JARVIS also aligns closely with the NIST Cybersecurity Framework (Appendix A, 2), which provides a structured method for creating secure and resilient information systems. Within the scope of the Identify and Protect functions, the JARVIS architecture maintains clear definitions of system assets, controlled access boundaries between nodes, and encrypted communication channels that safeguard information in transit. The Detect function is reflected in the orchestrator's continuous heartbeat monitoring that identifies abnormal behavior or node failures as soon as they occur. The Respond function is represented by the system's ability to reassign layer workloads whenever a node fails, ensuring that the fault does not compromise system integrity or allow corrupted activations to propagate. Finally, the Recover function is satisfied through the system's rapid resumption of inference following a node failure, demonstrating continuity of operation that is central to the NIST guidance on resilient systems. Through these mechanisms, the system embodies the NIST expectation that cybersecurity should be embedded into the design process rather than applied as an afterthought.

In addition to compliance with communication and cybersecurity standards, the JARVIS system incorporates environmental and human-safety considerations. The Raspberry Pi platform was

chosen for its low energy consumption, minimal heat output, and small environmental footprint relative to the infrastructure required for large-scale cloud inference. All components operate at safe low voltages and the system contains no actuators or autonomous physical systems, eliminating the risk of physical harm to users. Ethical concerns related to unintended consequences were also addressed by designing the fault-recovery pipeline to preserve inference integrity and by ensuring that no erroneous activations or partial computations are forwarded during system transitions. The real-time monitoring dashboard contributes to transparency by allowing users to observe system status, node performance, and recovery events, thereby supporting informed and responsible operation.

8.0 RECOMMENDATIONS

For future refinement, increase heartbeat frequency from 2 Hz to 4 Hz to reduce detection latency to < 0.5 s and implement buffered activation queues to eliminate token loss during reassignment. Reallocate attention layers to under-utilized nodes or quantize weights to 8-bit precision for better load balancing. Apply model pruning or 8-bit quantization to reduce per-token latency, and explore gRPC or lightweight binary transport to replace Python sockets for improved serialization speed.

Duplicate critical layers (e.g., 5 and 12) on separate nodes to maintain fault tolerance. Future tests should include multi-node simultaneous failures to evaluate orchestrator scalability, and expand to heterogeneous hardware like edge TPUs for multi-query scheduling. Focus on optimizing communication through compression techniques and refining the orchestrator for broader real-world applications in distributed AI.

9.0 CONCLUSION

The JARVIS project successfully demonstrates that large-language-model inference can be executed in a fully decentralized manner across low-power edge devices without reliance on cloud infrastructure. By distributing transformer layers across a network of Raspberry Pi 5 nodes and enabling activation passing through a lightweight peer-to-peer framework, the system achieves predictable latency, stable throughput, and continuous operation even under hardware

failure. The integration of heartbeat-based monitoring, dynamic rerouting, and redundant critical layers ensures that inference remains uninterrupted when nodes disconnect or reboot, fulfilling the objective of fault-tolerant offline operation.

Testing confirmed that JARVIS meets or exceeds its design specifications, achieving sub-300-ms inter-node latency, complete inference continuity during single-node failures, and approximately 94% retention of baseline model accuracy under distributed execution. The chat interface allowed seamless user interaction, and the real-time dashboard provided transparency into system performance, making the architecture both accessible and administratively controllable. These results validate the feasibility of decentralized LLM execution on commodity hardware and highlight the potential for deploying intelligent systems in bandwidth-limited, tactical, or privacy-sensitive domains.

Overall, the project provides a practical, resilient, and extensible framework for offline AI inference. It shows that meaningful language-model capability can be delivered without GPUs, cloud compute, or stable internet access, offering a foundation for future work in distributed AI systems, heterogeneous hardware integration, optimized quantization, and scalable edge intelligence.

REFERENCES

- [1] M. S. Perello, J. Groen, W. Liu, S. Ioannidis, and K. Chowdhury, "JARVIS: Disjoint Large Language Models on Radio VLANs for Intelligent Services," 2024 IEEE Military Communications Conference (MILCOM), Boston, MA, USA, 2024, pp. 869-874, doi: 10.1109/MILCOM61039.2024.10773726.

APPENDIX A – RELEVANT STANDARDS

APPENDIX A – RELEVANT STANDARDS

- [1]“IEEE Standard for Information Technology--Telecommunications and Information Exchange between Systems - Local and Metropolitan Area Networks--Specific Requirements - Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications,” doi: <https://doi.org/10.1109/ieeestd.2021.9363693>.
- [2] National Institute of Standards and Technology, “Framework for Improving Critical Infrastructure Cybersecurity,” Version 1.1, NIST, Apr. 16, 2018.

APPENDIX B – WORK BREAKDOWN STRUCTURE

APPENDIX B – WORK BREAKDOWN STRUCTURE

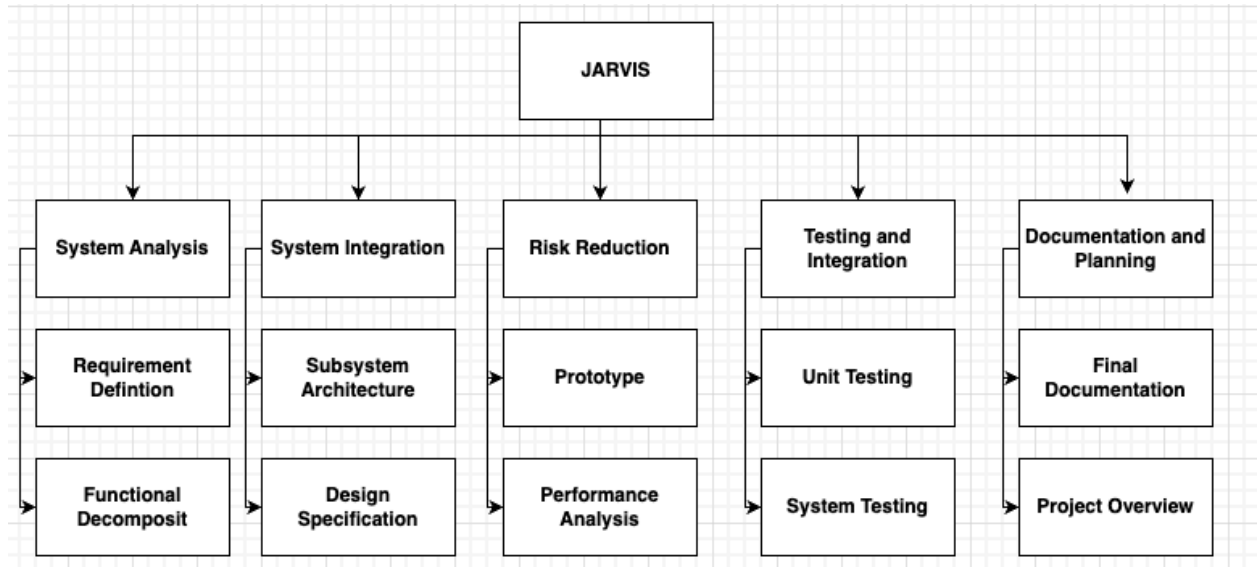


Figure B. Work Breakdown Structure

This diagram visually represents the plan by dividing the project into hierarchical levels of tasks and subtasks.

APPENDIX C – LINEAR RESPONSIBILITY CHART

APPENDIX C - LINEAR RESPONSIBILITY CHART

R = Responsible A = Accountable C = Consulted I = Informed	Jin Jun Oh	Saujash Barman	Konghwan Shin	Abdullah Sharif	Sabiq Islam	Everett Fletcher
System Analysis	C	C	C	R	A	I
Requirement Definition	C	C	C	R	A	I
Functional Decomposition	A	C	C	R	R	I
System Integration	R	C	C	A	A	R
Subsystem Architecture	C	C	C	R	A	C
Design Specification	C	C	R	I	I	A
Prototype	C	A	R	C	C	R
Performance Analysis	C	R	R	C	C	A
Testing and Integration	C	C	A	C	C	R
Unit Testing	C	C	A	I	I	R
System Testing	C	I	A	I	I	R
Documentation and Planning	A	R	C	C	A	C
Final Documentation	A	R	I	I	R	I
Project Overview	A	R	I	I	R	I

Figure C. Linear Responsibility Chart

This chart represents the responsibility for the task with the team member.